

# Blackbox · Database Schema

Supabase project: **crkmhpfgrvcnmqgiekjb** · schema: **public**

This document captures the current state of the database used by the Blackbox storefront and admin dashboard: tables, columns, row-level security policies, views, database functions, triggers, and edge functions. All views use `security_invoker = true` and all functions pin `search_path = public`.

## Overview

|                               |    |
|-------------------------------|----|
| Tables                        | 23 |
| Views                         | 3  |
| Database functions / triggers | 18 |
| Edge functions                | 2  |
| Enum types                    | 1  |

# Tables

## cart\_items

Per-user shopping cart line items.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| user_id    | uuid       | YES  | —                 |
| product_id | uuid       | YES  | —                 |
| variant_id | uuid       | YES  | —                 |
| quantity   | integer    | NO   | —                 |
| created_at | timestampz | YES  | now()             |
| updated_at | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                  | Condition            |
|-----|-------------------------|----------------------|
| ALL | Users manage their cart | auth.uid() = user_id |

## customers

Staff-managed CRM records (walk-in / phone customers). Not linked to auth.users.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| name       | text       | NO   | —                 |
| email      | text       | YES  | —                 |
| phone      | text       | YES  | —                 |
| address    | text       | YES  | —                 |
| created_at | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                      | Condition                     |
|-----|-----------------------------|-------------------------------|
| ALL | Admins can manage customers | has_role(auth.uid(), 'admin') |

## email\_blocklist

Suppression list for bounced or blocked email addresses. Service-role only.

### Columns

| Column     | Type       | Null | Default      |
|------------|------------|------|--------------|
| id         | integer    | NO   | nextval(seq) |
| email      | text       | NO   | —            |
| reason     | text       | YES  | —            |
| created_at | timestampz | YES  | now()        |

### RLS policies

| Cmd | Policy                                       | Condition |
|-----|----------------------------------------------|-----------|
| —   | RLS enabled, no policies (service-role only) | —         |

## email\_logs

Audit trail of outbound email delivery attempts. Service-role only.

### Columns

| Column      | Type       | Null | Default           |
|-------------|------------|------|-------------------|
| id          | uuid       | NO   | gen_random_uuid() |
| user_id     | uuid       | YES  | —                 |
| to_email    | text       | NO   | —                 |
| event_type  | text       | NO   | —                 |
| provider    | text       | NO   | —                 |
| provider_id | text       | YES  | —                 |
| status_code | integer    | YES  | —                 |
| error       | text       | YES  | —                 |
| attempt     | smallint   | YES  | 0                 |
| created_at  | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                                       | Condition |
|-----|----------------------------------------------|-----------|
| —   | RLS enabled, no policies (service-role only) | —         |

## email\_send\_queue

Queue of pending outbound email payloads. Service-role only.

### Columns

| Column        | Type       | Null | Default           |
|---------------|------------|------|-------------------|
| id            | uuid       | NO   | gen_random_uuid() |
| payload       | jsonb      | NO   | —                 |
| provider_hint | text       | YES  | —                 |
| state         | text       | YES  | 'pending'         |
| attempts      | smallint   | YES  | 0                 |
| last_error    | text       | YES  | —                 |
| scheduled_at  | timestampz | YES  | now()             |
| created_at    | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                                       | Condition |
|-----|----------------------------------------------|-----------|
| —   | RLS enabled, no policies (service-role only) | —         |

## inventory\_adjustments

Stock movements (restock, damage, return, correction, sale). Trigger applies the delta to products / product\_variants.

### Columns

| Column          | Type       | Null | Default           |
|-----------------|------------|------|-------------------|
| id              | uuid       | NO   | gen_random_uuid() |
| product_id      | uuid       | NO   | —                 |
| variant_id      | uuid       | YES  | —                 |
| adjustment_type | text       | NO   | —                 |
| quantity_change | integer    | NO   | —                 |
| reason          | text       | YES  | —                 |
| adjusted_by     | uuid       | YES  | —                 |
| created_at      | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                                  | Condition                     |
|-----|-----------------------------------------|-------------------------------|
| ALL | Admins can manage inventory adjustments | has_role(auth.uid(), 'admin') |

## message\_threads

Reply chain on a message. sender = 'admin' | 'customer'.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| message_id | uuid       | NO   | —                 |
| sender     | text       | NO   | —                 |
| content    | text       | NO   | —                 |
| created_at | timestampz | NO   | now()             |

### RLS policies

| Cmd | Policy                | Condition                     |
|-----|-----------------------|-------------------------------|
| ALL | Admins manage threads | has_role(auth.uid(), 'admin') |

## messages

Public contact-form submissions. display\_id format: MSG00001.

### Columns

| Column       | Type       | Null | Default            |
|--------------|------------|------|--------------------|
| id           | uuid       | NO   | gen_random_uuid()  |
| display_id   | text       | YES  | MSG##### (trigger) |
| sender_name  | text       | NO   | —                  |
| sender_email | text       | NO   | —                  |
| subject      | text       | NO   | —                  |
| body         | text       | NO   | ' '                |
| status       | text       | NO   | 'unseen'           |
| created_at   | timestampz | NO   | now()              |
| updated_at   | timestampz | NO   | now()              |

### RLS policies

| Cmd    | Policy                     | Condition                     |
|--------|----------------------------|-------------------------------|
| INSERT | Public can submit messages | validated name/email/subject  |
| SELECT | Admins manage messages     | has_role(auth.uid(), 'admin') |
| UPDATE | Admins update messages     | has_role(auth.uid(), 'admin') |
| DELETE | Admins delete messages     | has_role(auth.uid(), 'admin') |

## order\_items

Snapshot of products at the time of purchase. Triggers recalc orders.total\_price.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| order_id   | uuid       | YES  | —                 |
| product_id | uuid       | YES  | —                 |
| variant_id | uuid       | YES  | —                 |
| quantity   | integer    | YES  | —                 |
| price      | numeric    | YES  | —                 |
| unit_price | numeric    | YES  | —                 |
| created_at | timestampz | YES  | now()             |
| updated_at | timestampz | YES  | now()             |

### RLS policies

| Cmd    | Policy                                 | Condition                     |
|--------|----------------------------------------|-------------------------------|
| SELECT | Users can view their order items       | via orders.user_id            |
| INSERT | Users can insert their own order items | via orders.user_id            |
| SELECT | Admins can view all order items        | has_role(auth.uid(), 'admin') |

## orders

Customer orders. display\_id format: ORD00001. Tracking BLK-YYYYMMDD-xxxxxxx.

### Columns

| Column             | Type       | Null | Default             |
|--------------------|------------|------|---------------------|
| id                 | uuid       | NO   | gen_random_uuid()   |
| display_id         | text       | YES  | ORD#### (trigger)   |
| user_id            | uuid       | YES  | —                   |
| customer_id        | uuid       | YES  | —                   |
| status             | text       | YES  | 'pending'           |
| payment_status     | text       | YES  | 'pending'           |
| payment_method     | text       | YES  | —                   |
| total_amount       | numeric    | YES  | —                   |
| total_price        | numeric    | YES  | —                   |
| shipping_address   | text       | YES  | —                   |
| shipping_method    | text       | YES  | 'Standard Delivery' |
| shipping_cost      | numeric    | YES  | 0                   |
| tracking_number    | text       | YES  | —                   |
| estimated_delivery | timestampz | YES  | —                   |
| actual_delivery    | timestampz | YES  | —                   |
| created_at         | timestampz | YES  | now()               |
| updated_at         | timestampz | YES  | now()               |

### RLS policies

| Cmd    | Policy                            | Condition                     |
|--------|-----------------------------------|-------------------------------|
| SELECT | Users can view their orders       | auth.uid() = user_id          |
| INSERT | Users can insert their own orders | auth.uid() = user_id          |
| SELECT | Admins can view all orders        | has_role(auth.uid(), 'admin') |
| UPDATE | Admins can update orders          | has_role(auth.uid(), 'admin') |

## product\_variants

Per-product variant SKUs (color / RAM / storage) with price modifier and stock.

### Columns

| Column         | Type        | Null | Default           |
|----------------|-------------|------|-------------------|
| id             | uuid        | NO   | gen_random_uuid() |
| product_id     | uuid        | NO   | —                 |
| color          | text        | YES  | —                 |
| ram            | text        | YES  | —                 |
| storage        | text        | YES  | —                 |
| price_modifier | numeric     | YES  | 0                 |
| stock          | integer     | YES  | 0                 |
| sku            | text        | YES  | —                 |
| created_at     | timestamptz | YES  | now()             |
| updated_at     | timestamptz | YES  | now()             |

### RLS policies

| Cmd    | Policy                                    | Condition                     |
|--------|-------------------------------------------|-------------------------------|
| SELECT | Product variants are viewable by everyone | true                          |
| ALL    | Admins can manage product variants        | has_role(auth.uid(), 'admin') |



## products

Product catalog. rating/review\_count maintained by reviews trigger.

### Columns

| Column       | Type       | Null | Default           |
|--------------|------------|------|-------------------|
| id           | uuid       | NO   | gen_random_uuid() |
| name         | text       | NO   | —                 |
| description  | text       | YES  | —                 |
| price        | numeric    | NO   | —                 |
| stock        | integer    | YES  | 0                 |
| category     | text       | YES  | —                 |
| brand        | text       | YES  | —                 |
| model        | text       | YES  | —                 |
| sku          | text       | YES  | —                 |
| condition    | text       | YES  | 'new'             |
| status       | text       | YES  | 'active'          |
| image_url    | text       | YES  | —                 |
| rating       | numeric    | YES  | 4.5               |
| review_count | integer    | YES  | 0                 |
| discount     | integer    | YES  | 0                 |
| is_new       | boolean    | YES  | false             |
| featured     | boolean    | YES  | false             |
| colors       | text[]     | YES  | {}                |
| specs        | text[]     | YES  | {}                |
| created_at   | timestampz | YES  | now()             |
| updated_at   | timestampz | YES  | now()             |

### RLS policies

| Cmd    | Policy                            | Condition                     |
|--------|-----------------------------------|-------------------------------|
| SELECT | Products are viewable by everyone | true                          |
| ALL    | Admins can manage products        | has_role(auth.uid(), 'admin') |

## profiles

Public user profile. PK = auth.users.id. Auto-created by handle\_new\_user trigger.

### Columns

| Column            | Type        | Null | Default         |
|-------------------|-------------|------|-----------------|
| id                | uuid        | NO   | = auth.users.id |
| email             | text        | YES  | —               |
| full_name         | text        | YES  | —               |
| phone             | text        | YES  | —               |
| address           | text        | YES  | —               |
| city              | text        | YES  | —               |
| region            | text        | YES  | —               |
| is_suspended      | boolean     | YES  | false           |
| suspended_at      | timestamptz | YES  | —               |
| suspension_reason | text        | YES  | —               |
| created_at        | timestamptz | YES  | now()           |
| updated_at        | timestamptz | YES  | now()           |

### RLS policies

| Cmd    | Policy                              | Condition                                       |
|--------|-------------------------------------|-------------------------------------------------|
| SELECT | Users can view own profile          | auth.uid() = id                                 |
| INSERT | Users can insert own profile        | auth.uid() = id                                 |
| UPDATE | Users can update own profile safely | auth.uid() = id AND suspension fields unchanged |
| SELECT | Admins can view all profiles        | has_role(auth.uid(), 'admin')                   |
| UPDATE | Admins can update all profiles      | has_role(auth.uid(), 'admin')                   |

## purchase\_order\_items

Line items on a supplier purchase order.

### Columns

| Column            | Type        | Null | Default           |
|-------------------|-------------|------|-------------------|
| id                | uuid        | NO   | gen_random_uuid() |
| purchase_order_id | uuid        | NO   | —                 |
| product_id        | uuid        | NO   | —                 |
| variant_id        | uuid        | YES  | —                 |
| quantity          | integer     | NO   | 0                 |
| unit_cost         | numeric     | YES  | 0                 |
| created_at        | timestamptz | YES  | now()             |

### RLS policies

| Cmd | Policy                                 | Condition                     |
|-----|----------------------------------------|-------------------------------|
| ALL | Admins can manage purchase order items | has_role(auth.uid(), 'admin') |

## **purchase\_orders**

Inbound stock orders placed with suppliers.

### **Columns**

| Column      | Type       | Null | Default           |
|-------------|------------|------|-------------------|
| id          | uuid       | NO   | gen_random_uuid() |
| supplier_id | uuid       | YES  | —                 |
| status      | text       | YES  | 'pending'         |
| total_cost  | numeric    | YES  | 0                 |
| notes       | text       | YES  | —                 |
| ordered_by  | uuid       | YES  | —                 |
| created_at  | timestampz | YES  | now()             |
| updated_at  | timestampz | YES  | now()             |

### **RLS policies**

| Cmd | Policy                            | Condition                     |
|-----|-----------------------------------|-------------------------------|
| ALL | Admins can manage purchase orders | has_role(auth.uid(), 'admin') |

## repair\_requests

Customer repair tickets. display\_id REP00001. Auth-only (no plaintext device password).

### Columns

| Column             | Type       | Null | Default           |
|--------------------|------------|------|-------------------|
| id                 | uuid       | NO   | gen_random_uuid() |
| display_id         | text       | YES  | REP#### (trigger) |
| user_id            | uuid       | YES  | —                 |
| customer_id        | uuid       | YES  | —                 |
| user_name          | text       | YES  | ' '               |
| device_brand       | text       | YES  | —                 |
| device_model       | text       | YES  | —                 |
| issue_type         | text       | YES  | —                 |
| issue_description  | text       | YES  | —                 |
| ai_diagnosis       | text       | YES  | —                 |
| image_url          | text       | YES  | —                 |
| urgency            | text       | YES  | 'standard'        |
| fulfillment_method | text       | YES  | 'Headquarters'    |
| preferred_date     | date       | YES  | —                 |
| preferred_time     | text       | YES  | —                 |
| contact_name       | text       | YES  | —                 |
| contact_email      | text       | YES  | —                 |
| contact_phone      | text       | YES  | —                 |
| diagnostic_fee     | text       | YES  | 'waived'          |
| repair_approval    | text       | YES  | 'quote'           |
| data_backup        | text       | YES  | 'acknowledges'    |
| agrees_to_terms    | boolean    | YES  | false             |
| client_signature   | text       | YES  | —                 |
| estimated_cost     | numeric    | YES  | —                 |
| final_cost         | numeric    | YES  | —                 |
| technician_notes   | text       | YES  | —                 |
| admin_note         | text       | YES  | —                 |
| status             | text       | YES  | 'pending'         |
| created_at         | timestampz | YES  | now()             |
| updated_at         | timestampz | YES  | now()             |

### RLS policies

| Cmd    | Policy                      | Condition                                    |
|--------|-----------------------------|----------------------------------------------|
| SELECT | Users view their repairs    | auth.uid() = user_id                         |
| INSERT | Users create their repairs  | auth.uid() = user_id AND user_id IS NOT NULL |
| UPDATE | Users update their repairs  | auth.uid() = user_id                         |
| DELETE | Users delete their repairs  | auth.uid() = user_id                         |
| SELECT | Admins can view all repairs | has_role(auth.uid(), 'admin')                |
| UPDATE | Admins can update repairs   | has_role(auth.uid(), 'admin')                |

## reviews

Product reviews. Trigger updates products.rating and review\_count.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| product_id | uuid       | NO   | —                 |
| user_id    | uuid       | YES  | —                 |
| user_name  | text       | NO   | 'Anonymous '      |
| rating     | integer    | NO   | —                 |
| title      | text       | YES  | —                 |
| body       | text       | YES  | —                 |
| created_at | timestampz | NO   | now()             |

### RLS policies

| Cmd    | Policy                   | Condition                     |
|--------|--------------------------|-------------------------------|
| SELECT | Anyone reads reviews     | true                          |
| INSERT | Users write own reviews  | auth.uid() = user_id          |
| UPDATE | Users update own reviews | auth.uid() = user_id          |
| ALL    | Admins manage reviews    | has_role(auth.uid(), 'admin') |

## signup\_attempts

Abuse-monitoring log for signup attempts (email, IP, UA). Service-role only.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| email      | text       | NO   | —                 |
| ip         | inet       | YES  | —                 |
| user_agent | text       | YES  | —                 |
| reason     | text       | YES  | —                 |
| metadata   | jsonb      | YES  | —                 |
| created_at | timestampz | NO   | now()             |

### RLS policies

| Cmd | Policy                                       | Condition |
|-----|----------------------------------------------|-----------|
| —   | RLS enabled, no policies (service-role only) | —         |

## suppliers

Vendor catalog for purchase orders.

### Columns

| Column        | Type       | Null | Default           |
|---------------|------------|------|-------------------|
| id            | uuid       | NO   | gen_random_uuid() |
| name          | text       | NO   | —                 |
| contact_email | text       | YES  | —                 |
| contact_phone | text       | YES  | —                 |
| address       | text       | YES  | —                 |
| notes         | text       | YES  | —                 |
| created_at    | timestampz | YES  | now()             |
| updated_at    | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                      | Condition                     |
|-----|-----------------------------|-------------------------------|
| ALL | Admins can manage suppliers | has_role(auth.uid(), 'admin') |

## tracking\_updates

Append-only chronological history of order status changes (auto-inserted by trigger).

### Columns

| Column      | Type       | Null | Default           |
|-------------|------------|------|-------------------|
| id          | uuid       | NO   | gen_random_uuid() |
| order_id    | uuid       | NO   | —                 |
| status      | text       | NO   | —                 |
| description | text       | YES  | —                 |
| location    | text       | YES  | —                 |
| created_at  | timestampz | YES  | now()             |

### RLS policies

| Cmd    | Policy                                | Condition                     |
|--------|---------------------------------------|-------------------------------|
| SELECT | Users can view their tracking updates | via orders.user_id            |
| ALL    | Admins can manage tracking updates    | has_role(auth.uid(), 'admin') |

## trade\_in\_requests

Customer trade-in submissions. display\_id TRD00001.

### Columns

| Column             | Type        | Null | Default           |
|--------------------|-------------|------|-------------------|
| id                 | uuid        | NO   | gen_random_uuid() |
| display_id         | text        | YES  | TRD#### (trigger) |
| user_id            | uuid        | YES  | —                 |
| customer_id        | uuid        | YES  | —                 |
| user_name          | text        | YES  | ' '               |
| user_email         | text        | YES  | —                 |
| device_brand       | text        | YES  | —                 |
| device_name        | text        | YES  | —                 |
| condition          | text        | YES  | —                 |
| description        | text        | YES  | —                 |
| user_description   | text        | YES  | —                 |
| target_device      | text        | YES  | —                 |
| estimated_value    | numeric     | YES  | 0                 |
| offered_price      | numeric     | YES  | —                 |
| final_value        | numeric     | YES  | —                 |
| fulfillment_method | text        | YES  | 'Headquarters'    |
| preferred_date     | date        | YES  | —                 |
| preferred_time     | text        | YES  | —                 |
| contact_name       | text        | YES  | —                 |
| contact_email      | text        | YES  | —                 |
| contact_phone      | text        | YES  | —                 |
| admin_notes        | text        | YES  | —                 |
| status             | text        | YES  | 'submitted'       |
| created_at         | timestamptz | YES  | now()             |
| updated_at         | timestamptz | YES  | now()             |

### RLS policies

| Cmd    | Policy                        | Condition                     |
|--------|-------------------------------|-------------------------------|
| ALL    | Users manage their trade-ins  | auth.uid() = user_id          |
| SELECT | Admins can view all trade-ins | has_role(auth.uid(), 'admin') |
| UPDATE | Admins can update trade-ins   | has_role(auth.uid(), 'admin') |

## user\_roles

Role assignments. Each user has at most one row (enforced by trigger). Roles: admin, moderator, user.

### Columns

| Column  | Type     | Null | Default           |
|---------|----------|------|-------------------|
| id      | uuid     | NO   | gen_random_uuid() |
| user_id | uuid     | NO   | —                 |
| role    | app_role | NO   | —                 |

### RLS policies

| Cmd    | Policy                       | Condition                     |
|--------|------------------------------|-------------------------------|
| SELECT | Users can view own roles     | auth.uid() = user_id          |
| ALL    | Admins can manage user roles | has_role(auth.uid(), 'admin') |

## wishlist

User-saved products.

### Columns

| Column     | Type       | Null | Default           |
|------------|------------|------|-------------------|
| id         | uuid       | NO   | gen_random_uuid() |
| user_id    | uuid       | NO   | —                 |
| product_id | uuid       | NO   | —                 |
| created_at | timestampz | YES  | now()             |

### RLS policies

| Cmd | Policy                      | Condition            |
|-----|-----------------------------|----------------------|
| ALL | Users manage their wishlist | auth.uid() = user_id |



# Views

| Name               | Description                                                                             |
|--------------------|-----------------------------------------------------------------------------------------|
| admin_orders_view  | Flat order list joined with profile name/email and item count. security_invoker = true. |
| revenue_summary    | Per-day order count, gross revenue, and delivered revenue. security_invoker = true.     |
| low_stock_products | Products with stock < 5, ordered ascending. security_invoker = true.                    |

# Enum types

| Name     | Values                 |
|----------|------------------------|
| app_role | admin, moderator, user |

# Database functions & triggers

| Name                                                                                                  | Mode             | Purpose                                                                                                                            |
|-------------------------------------------------------------------------------------------------------|------------------|------------------------------------------------------------------------------------------------------------------------------------|
| has_role(uuid, app_role) → boolean                                                                    | SECURITY DEFINER | Returns true if the user has the role. Used inside RLS policies.                                                                   |
| handle_new_user() trigger                                                                             | SECURITY DEFINER | Creates a public.profiles row when a new auth.users row is inserted.                                                               |
| place_order(p_cart_items jsonb) → uuid                                                                | SECURITY DEFINER | Validates auth + suspension, computes totals server-side, creates order, items and the initial tracking row, then clears the cart. |
| update_order_status(uuid, text)                                                                       | SECURITY DEFINER | Admin-only. Validates status enum and updates the order. Trigger appends a tracking_updates row.                                   |
| update_repair_status(uuid, text)                                                                      | SECURITY DEFINER | Admin-only repair status transition with enum validation.                                                                          |
| update_trade_in(uuid, text, numeric)                                                                  | SECURITY DEFINER | Admin-only trade-in status / offered price update.                                                                                 |
| suspend_user(uuid, text)                                                                              | SECURITY DEFINER | Admin-only. Sets is_suspended + reason; the next auth refresh signs the user out.                                                  |
| reactivate_user(uuid)                                                                                 | SECURITY DEFINER | Admin-only. Clears suspension fields.                                                                                              |
| admin_delete_user(uuid)                                                                               | SECURITY DEFINER | Admin-only. Removes profile + role rows. The auth account is deleted by the admin-delete-user edge function.                       |
| apply_inventory_adjustment() trigger                                                                  | SECURITY DEFINER | Applies quantity_change to product_variants.stock or products.stock when an inventory_adjustments row is inserted.                 |
| recalculate_order_total() trigger                                                                     | SECURITY DEFINER | Keeps orders.total_price in sync with the sum of its order_items.                                                                  |
| recalculate_product_rating() trigger                                                                  | trigger          | Updates products.rating (avg) and review_count when reviews change.                                                                |
| track_order_status_change() trigger                                                                   | trigger          | Appends a tracking_updates row whenever orders.status changes.                                                                     |
| repair_estimate_status() trigger                                                                      | trigger          | When estimated_cost is first set, advances repair_requests.status to 'estimate_sent'.                                              |
| trade_offer_status() trigger                                                                          | trigger          | When final_value is first set, advances trade_in_requests.status to 'offer_made'.                                                  |
| set_order_display_id / set_repair_display_id / set_trade_display_id / set_message_display_id triggers | trigger          | Generate human-readable IDs (ORD#####, REP#####, TRD#####, MSG#####,) via generate_readable_id + dedicated sequences.              |
| prevent_duplicate_user_roles() trigger                                                                | SECURITY DEFINER | Enforces one role per user at write time.                                                                                          |
| set_updated_at() trigger                                                                              | trigger          | Generic updated_at = now() helper.                                                                                                 |

## Edge functions

| Name              | Purpose                                                                           |
|-------------------|-----------------------------------------------------------------------------------|
| admin-delete-user | Deletes the Supabase auth account after admin_delete_user clears the public rows. |
| delete-account    | Lets a signed-in user delete their own account (auth + public data).              |

Generated from the live Supabase project schema. Tables in the auth, storage, realtime, supabase\_functions and vault schemas are managed by Supabase and are not listed here.